

Nesneye Yönelik Programlama

Teori 1 — Miras Alma (Inheritance)

Bir sınıfın başka bir sınıfın özelliklerini (alan + metot) devralmasıdır.

```
// Üst sınıf (parent / superclass)
class Hayvan {
    String isim;
    public void yemekYe() {
        System.out.println(isim + " yemek yiyor.");
    }
}

// Alt sınıf (child / subclass)
class Kopek extends Hayvan {
    public void havla() {
        System.out.println(isim + " hav hav!"); // isim devralındı
    }
}
```

Kopek IS-A Hayvan → extends ile "bir tür ... dir" ilişkisi kurulur.

Teori: Miras Alma (Inheritance)

Ne işe yarar?

- Ortak özelliklere sahip sınıflar arasında kodu tek bir yerde yazmanızı sağlar.
- Alt sınıf (child), ata sınıfın (parent) public ve protected üyelerini miras alır.
- Java'da sadece TEK bir sınıftan miras alınabilir (multiple inheritance YOK).
- "is-a" ilişkisi için kullanılır: Player IS-A GameEntity, Admin IS-A User.
- Sınıf hiyerarşisi özgülden genele doğru yukarı çıkar: Object sınıfı her sınıfın atasıdır.

```
class Character {
    protected String name;
    protected int hp;

    public void move() {
        System.out.println(name + " hareket etti.");
    }
}

// Character'dan miras alıyor
class Mage extends Character {
    private int mana;

    public void castSpell() {
        System.out.println(name + " büyü yaptı!");
    }
}
```

Teori 3 — Method Overriding

Alt sınıfın, üst sınıftan miras aldığı bir metodu AYNI imzayla yeniden yazmasıdır.

```
class Sekil {  
    public double alan() { return 0.0; }  
}  
  
class Daire extends Sekil {  
    double yaricap;  
    @Override // zorunlu değil ama ÖNERİLİR  
    public double alan() { // aynı isim + parametre + dönüş tipi  
        return Math.PI * yaricap * yaricap;  
    }  
}
```

- Metot imzası (isim + parametre listesi) birebir AYNI olmalı.
- @Override etiketi, derleyiciye kontrol ettirir — yazım hatasını yakalar.
- Çalışma zamanında dinamik bağlama (polymorphism) ile çözülür.

Teori 4 — Method Overloading

Aynı sınıfta AYNI isimli birden fazla metot tanımlamaktır — parametre listeleri farklı olmalı.

```
class Hesaplayici {  
    public int toplama(int a, int b)           { return a + b;      }  
    public double toplama(double a, double b) { return a + b;      }  
    public int toplama(int a, int b, int c)    { return a + b + c;   }  
    public String toplama(String a, String b)  { return a + b;      }  
}
```

```
// Derleyici parametrelere bakarak hangi metodu çağıracağına  
// DERLEME zamanında karar verir → static binding.  
Hesaplayici h = new Hesaplayici();  
h.toplama(2, 3);           // int versiyonu  
h.toplama(2.5, 3.1);       // double versiyonu  
h.toplama("a", "b");       // String versiyonu
```

Soru 2 — İlk Miras Alma

- Hayvan sınıfı oluşturun:
 - alanlar: String isim, int yas
 - metotlar: yemekYe(), uyu() — her biri isim ile bir mesaj yazdırsın
- Kopek sınıfı yazın — Hayvan'dan miras alsın:
 - sadece yeni bir metot ekleyin: havla()
- main metodunda bir Kopek nesnesi oluşturup isim ve yaş atayın.
- yemekYe, uyu ve havla metotlarının üçünü de çağırın.

Kopek'te sadece havla() var, peki yemekYe() nasıl çalışıyor?

Soru 2 — Şimdi Siz Deneyin!

İpuçları

- `class Kopek extends Hayvan { ... }` → `extends` anahtar kelimesini unutmayın.
- Hayvan'daki alanları Kopek içinde TEKRAR tanımlamayın — otomatik devralınır.
- Kopek nesnesi, Hayvan'ın tüm public metotlarını doğrudan çağırabilir.
- Alan erişimini de deneyin: `kopek.isim = "Karabaş";`
- Sonuç: Az kod ile tekrar kullanım → kalıtımın temel faydası.

```
class Hayvan {
    String isim;
    int yas;
    public void yemekYe() {
        System.out.println(isim + " yemek yiyor.");
    }
    public void uyu() {
        System.out.println(isim + " uyuyor.");
    }
}

class Kopek extends Hayvan {           // Hayvan'ın her şeyini devralır
    public void havla() {
        System.out.println(isim + " hav hav!");
    }
}

public class Main {
    public static void main(String[] args) {
        Kopek karabas = new Kopek();
        karabas.isim = "Karabaş";      // Hayvan'dan gelen alan
        karabas.yas = 3;
        karabas.yemekYe();              // Hayvan'dan miras
        karabas.uyu();                  // Hayvan'dan miras
        karabas.havla();                 // Kopek'e özel
    }
}
```


Soru 3 — super() ile Constructor Çağırma

- Calisan sınıfı yazın:
 - alanlar: String ad, double maas
 - constructor: ad ve maas parametre alsın, ekrana "Calisan constructor çalıştı" yazsın
 - metot: bilgiYazdir() — ad ve maas bilgisini yazdırsın
- Muhendis sınıfı — Calisan'dan miras alsın:
 - ek alan: String uzmanlikAlani
 - constructor: üç parametre (ad, maas, uzmanlik) alıp super(ad, maas) çağırınsın
- main: Muhendis oluşturun, bilgiYazdir() çağırın, uzmanlığı da yazdırın.

Dikkat: super(...) çağırısı constructor'ın İLK satırı olmalıdır.

Soru 3

İpuçları

- `super(ad, maas)` → üst sınıfın constructor'ını parametrelerle çağırır.
- Bu çağrıyı UNUTURSANIZ Java default (parametresiz) constructor arar — hata verebilir.
- Kendi constructor'ınızda önce `super(...)` sonra kendi alanlarınızı atayın.
- Çıktıya bakın: önce Calisan sonra Muhendis mesajı yazılıyor mu?
- Bu, nesne oluşturulurken kurucuların miras zincirinde ÇALIŞMA SIRASıdır.

```
class Calisan {
String ad;
double maas;
public Calisan(String ad, double maas) {
this.ad = ad;
this.maas = maas;
System.out.println("Calisan constructor çalıştı.");}
public void bilgiYazdir() {
System.out.print("Ad: " + ad + ", Maaş: " + maas);}
}
class Muhendis extends Calisan {
String uzmanlikAlani;
public Muhendis(String ad, double maas, String uzmanlikAlani) {
super(ad, maas);
this.uzmanlikAlani = uzmanlikAlani;
System.out.println("Muhendis constructor çalıştı.");
}
@Override
public void bilgiYazdir() {
super.bilgiYazdir();
System.out.print(", Uzmanlık: " + uzmanlikAlani);
}
}
```

```
public class Main {
public static void main(String[] args) {
Muhendis m = new Muhendis("Ayşe", 45000, "Yazılım");
m.bilgiYazdir();}
}
```

Soru 4 — Method Override (Şekiller)

- **Konu: Override + Polimorfizm**
- Sekil sınıfı yazın:
 - metot: double alan() → varsayılan olarak 0.0 döndürsün
 - metot: void bilgiYazdir() → "Alan: " + alan() yazdırsın
- Daire sınıfı — Sekil'den miras alsın:
 - alan: double yaricap (constructor ile verilsin)
 - alan() metodunu OVERRIDE edin: $\pi \cdot r^2$
- Kare sınıfı — Sekil'den miras alsın:
 - alan: double kenar (constructor ile verilsin)
 - alan() metodunu OVERRIDE edin: kenar · kenar
- main: Sekil tipinde referanslarla Daire ve Kare oluşturup bilgiYazdir() çağırın.

Soru 4

İpuçları

- Override için imza (isim + parametre + dönüş tipi) AYNI olmalı.
- @Override etiketini kullanın — yazım hatası olursa derleyici size söyler.
- Math.PI sabitini kullanın.
- Ana noktayı fark edin: Sekil `s = new Daire(5); s.alan()` çağrıldığında `Daire`'nin `alan()`'ı çalışır.
- Bu davranışın adı: DİNAMİK METOT BAĞLAMA (polymorphism).

```

class Sekil {
    public double alan() {
        return 0.0;
    }
    public void bilgiYazdir() {
        System.out.println("Alan: " + alan());
    }
}
class Daire extends Sekil {
    double yaricap;
    public Daire(double yaricap) { this.yaricap = yaricap; }

    @Override
    public double alan() {
        return Math.PI * yaricap * yaricap;
    }
}

class Kare extends Sekil {
    double kenar;
    public Kare(double kenar) { this.kenar = kenar; }

    @Override
    public double alan() {
        return kenar * kenar;
    }
}

public class Main {
    public static void main(String[] args) {
        Sekil s1 = new Daire(5); // Sekil referansı, Daire nesnesi
        Sekil s2 = new Kare(4);
        s1.bilgiYazdir(); // Alan: 78.5398...
        s2.bilgiYazdir(); // Alan: 16.0
    }
}

```